Unidad: 1 Actividad entregable: Diseño seguro desde el requerimiento: Análisis y propuesta metodológica.

Diana Zoraida Romero Ramírez

ID: 949490.

Corporación Universitaria Minuto de Dios UNIMINUTO

Programa: Ingeniería de Sistemas

NRC: 40-63386 Desarrollo de software seguro.

Docente: Edwin Albeiro Ramos Villamil.

Marzo 29, 2025

Bogotá D.C

Tabla de contenido

1. Introducción……3
2. Análisis del caso con descripción del entorno, producto y contexto…….4
2.1 Descripción general del sistema……...4
2.2 Funcionalidades y tipos de usuario……..4
2.3 Entorno tecnológico…….4
2.4 Importancia del contexto en el sector salud……5
2.5 Vulnerabilidades identificadas en el sistema……...5
2.6 Análisis del enfoque de desarrollo……..5
2.7 Conclusión del análisis del caso……..6
3. Clasificación detallada de amenazas encontradas……...6
3.1 Enfoque de clasificación mediante STRIDE……...6
3.2 Análisis de amenazas identificadas……6
3.3 Valoración de riesgos……..8
4. Justificación de la metodología de desarrollo seguro seleccionada……..8
4.1 Análisis de metodologías de desarrollo seguro……8
4.2 Metodología seleccionada: DevSecOps……...9
4.3 Justificación de la elección frente al caso analizado……9
4.4 Ventajas de DevSecOps en el contexto del sistema……10
5. Plan de integración desde los requerimientos……10
5.1 Enfoque de integración de seguridad desde etapas tempranas……..10
5.2 Definición de requisitos de seguridad……...11
5.3 Integración de seguridad en la fase de diseño……...11
5.4 Aplicación de prácticas seguras en el desarrollo……...12
5.5 Pruebas de seguridad e integración continua……12
5.6 Seguridad en el despliegue y mantenimiento……12
6. Mapa de buenas prácticas por fase del ciclo de vida……13
6.1 Representación del mapa de buenas prácticas……...13
6.2 Análisis del mapa……...14
7. Matriz metodológica con riesgos, soluciones y fases afectadas……14
7.1 Enfoque de la matriz metodológica……14

7.2 Matriz de integración de riesgos y soluciones……14
7.3 Análisis de la matriz……...15
8. Conclusión…….17
9. Referencias……18

1. Introducción

En el desarrollo de software actual, la seguridad se ha convertido en un aspecto fundamental que debe ser considerado desde las primeras etapas del ciclo de vida. Sin embargo, en muchos proyectos aún se aborda como un elemento secundario, incorporándose únicamente cuando el sistema ya ha sido implementado, lo que incrementa el riesgo de vulnerabilidades y fallas críticas.

El presente trabajo tiene como objetivo analizar un caso de estudio relacionado con una plataforma web de gestión de citas médicas, en la cual se identifican diversas debilidades de seguridad derivadas de la ausencia de un enfoque preventivo desde la fase de requerimientos. A partir de este análisis, se realiza una clasificación de amenazas utilizando el modelo STRIDE, permitiendo comprender los riesgos asociados al sistema y su impacto dentro del contexto del sector salud.

Posteriormente, se selecciona una metodología de desarrollo seguro adecuada, en este caso DevSecOps, la cual permite integrar la seguridad de manera continua a lo largo de todo el ciclo de vida del software. Asimismo, se propone un plan de integración desde los requerimientos, acompañado de un mapa de buenas prácticas y una matriz metodológica que relaciona las amenazas identificadas con las soluciones planteadas.

De esta manera, el trabajo busca evidenciar la importancia de incorporar la seguridad como un componente transversal en el desarrollo de software, no solo para reducir riesgos técnicos, sino también para garantizar la protección de la información y la confiabilidad de los sistemas en entornos críticos.

2. Análisis del caso con descripción del entorno, producto y contexto

2.1 Descripción general del sistema

El caso de estudio seleccionado corresponde a una plataforma web orientada a la gestión de citas médicas e información clínica básica, desarrollada para una institución de salud de tamaño mediano que busca modernizar sus procesos de atención. El sistema fue concebido con el objetivo de facilitar la interacción entre pacientes, personal médico y administradores, permitiendo centralizar en una sola aplicación funciones como el registro de usuarios, autenticación, agendamiento de citas, consulta de horarios médicos y acceso a determinados datos clínicos.

2.2 Funcionalidades y tipos de usuario

Desde el punto de vista funcional, el producto ofrece diferentes módulos según el tipo de usuario. En el caso de los pacientes, la plataforma permite crear una cuenta, iniciar sesión, solicitar citas, consultar disponibilidad de especialistas y revisar información general asociada a su atención. Para el personal médico, el sistema incluye opciones para visualizar agendas, consultar historiales básicos y registrar observaciones relacionadas con la consulta. Finalmente, existe un perfil administrativo encargado de gestionar usuarios, disponibilidad de servicios, especialidades médicas y reportes operativos de la institución. Esto convierte al sistema en una aplicación crítica, ya que concentra procesos sensibles y datos que deben mantenerse protegidos en todo momento.

2.3 Entorno tecnológico

En cuanto al entorno tecnológico, se trata de una aplicación cliente-servidor accesible desde navegador web, conectada a una base de datos central y desplegada en infraestructura institucional. Aunque su arquitectura responde a un esquema funcional común en sistemas de información, el principal problema identificado es que el desarrollo se enfocó más en cumplir con requerimientos operativos de manera rápida que en establecer controles de seguridad desde el inicio. En otras palabras, el sistema fue construido priorizando la entrega de funcionalidades visibles, mientras que aspectos como autenticación segura, validación de entradas, control de acceso y trazabilidad de acciones quedaron en un segundo plano.

2.4 Importancia del contexto en el sector salud

Este contexto es relevante porque el sector salud trabaja con información altamente sensible, incluyendo datos personales, datos de contacto, antecedentes de atención y elementos asociados a la historia clínica del paciente. Cualquier debilidad en la aplicación puede comprometer principios fundamentales de seguridad como la confidencialidad, la integridad y la disponibilidad de la información. No se trata únicamente de un riesgo técnico, sino también de un problema que puede afectar la confianza de los usuarios, la continuidad del servicio y la responsabilidad institucional frente al manejo adecuado de los datos.

2.5 Vulnerabilidades identificadas en el sistema

Durante el análisis preliminar del caso se identificaron varias fallas representativas de un desarrollo sin enfoque de seguridad desde el requerimiento. Una de las principales debilidades se encuentra en el manejo de credenciales, ya que las contraseñas no fueron protegidas mediante mecanismos robustos de cifrado o hash. A esto se suman fallas de validación en formularios y parámetros de entrada, lo cual expone al sistema a ataques como inyección de código o manipulación de información. También se evidencian problemas de autorización, debido a que ciertos recursos pueden ser accedidos por usuarios sin los privilegios adecuados, ya sea por errores en la lógica de roles o por una incorrecta protección de rutas y servicios. De igual manera, se observan carencias en el control de sesiones, ausencia de registros de auditoría suficientemente detallados y una configuración deficiente en componentes que deberían estar restringidos.

2.6 Análisis del enfoque de desarrollo

Más allá de las vulnerabilidades puntuales, este caso refleja una situación común en muchos proyectos de software: La seguridad se aborda de forma reactiva, es decir, cuando ya existen errores, incidentes o hallazgos, y no como un criterio transversal de diseño y desarrollo. Esto hace que los costos de corrección sean mayores y que la exposición al riesgo aumente a medida que el sistema entra en operación. Por ello, el caso resulta apropiado para sustentar la importancia de integrar una metodología de desarrollo seguro

desde las primeras fases del ciclo de vida, especialmente desde el levantamiento y definición de requerimientos.

2.7 Conclusión del análisis del caso

En ese sentido, el análisis del entorno, del producto y del contexto permite concluir que la plataforma no presenta únicamente errores técnicos aislados, sino una debilidad estructural en la forma como fue concebida y desarrollada. Precisamente por eso, el estudio de este caso permite avanzar hacia una propuesta metodológica orientada a incorporar seguridad desde el requerimiento, establecer buenas prácticas por fase y reducir la probabilidad de vulnerabilidades en etapas posteriores del ciclo de vida del software.

3. Clasificación detallada de amenazas encontradas.

Con el propósito de organizar y analizar de forma estructurada las vulnerabilidades identificadas en la plataforma de citas médicas, se utiliza el modelo STRIDE, el cual permite clasificar las amenazas de seguridad en seis categorías principales. Este enfoque facilita no solo la identificación de los riesgos, sino también su comprensión dentro del contexto del sistema y su posterior tratamiento mediante una metodología de desarrollo seguro.

3.1 Enfoque de clasificación mediante STRIDE

El modelo STRIDE agrupa las amenazas en: Suplantación de identidad, manipulación de datos, repudio, divulgación de información, denegación de servicio y elevación de privilegios. Su aplicación en este caso permite evidenciar que las vulnerabilidades encontradas no son aisladas, sino que responden a la ausencia de controles de seguridad desde etapas tempranas del desarrollo.

3.2 Análisis de amenazas identificadas

Suplantación de identidad (Spoofing)

En el sistema analizado se identifican debilidades en los mecanismos de autenticación, especialmente relacionadas con el manejo de credenciales. La falta de protección adecuada de contraseñas, junto con la ausencia de políticas de seguridad robustas, facilita que un atacante pueda acceder a cuentas legítimas. Este tipo de amenaza

resulta crítico en el contexto del sistema, ya que permitiría a un tercero acceder a información médica sensible o realizar acciones en nombre de otro usuario.

Manipulación de información (Tampering)

Se evidencian fallas en la validación de entradas, lo cual permite la alteración de datos tanto en formularios como en parámetros enviados al servidor. Esta debilidad compromete la integridad de la información, ya que un usuario o atacante podría modificar registros de citas, datos personales o información clínica. La ausencia de validaciones adecuadas en el backend aumenta considerablemente este riesgo.

Repudio (Repudiation)

El sistema carece de mecanismos adecuados de auditoría y registro de actividades. No se cuenta con logs detallados que permitan rastrear las acciones realizadas por los usuarios, lo cual dificulta la identificación de responsables ante un incidente. Esta situación afecta la trazabilidad del sistema y limita la capacidad de respuesta ante eventos de seguridad.

Divulgación de información (Information Disclosure)

Una de las amenazas más críticas detectadas corresponde a la exposición de información sensible. Debido a fallas en el control de acceso y protección de recursos, es posible que usuarios accedan a datos que no les corresponden, como historiales médicos o información personal de otros pacientes. Este tipo de vulnerabilidad compromete directamente la confidencialidad del sistema y puede generar impactos significativos tanto a nivel técnico como institucional.

Denegación de servicio (Denial of Service)

Aunque no se identificaron mecanismos directos de ataque implementados, el sistema no cuenta con controles que limiten el número de solicitudes o gestionen adecuadamente los recursos. Esto lo hace vulnerable a ataques de saturación, en los cuales un atacante podría afectar la disponibilidad del servicio mediante múltiples peticiones simultáneas.

Elevación de privilegios (Elevation of Privilege)

Se detectan deficiencias en la gestión de roles y permisos dentro del sistema. La falta de una adecuada separación de privilegios permite que usuarios con permisos básicos accedan a funciones restringidas mediante la manipulación de rutas o parámetros. Esta vulnerabilidad evidencia una debilidad en el diseño de la arquitectura de seguridad.

3.3 Valoración de riesgos

Para complementar el análisis, se realiza una valoración de las amenazas identificadas considerando su impacto y probabilidad dentro del contexto del sistema:

Amenaza	Impacto	Probabilidad	Nivel de riesgo
Suplantación de identidad	Alto	Medio	Alto
Manipulación de datos	Alto	Alto	Crítico
Repudio	Medio	Medio	Medio
Divulgación de información	Alto	Alto	Crítico
Denegación de servicio	Medio	Medio	Medio
Elevación de privilegios	Alto	Medio	Alto

4. Justificación de la metodología de desarrollo seguro seleccionada.

4.1 Análisis de metodologías de desarrollo seguro

Con el fin de abordar las vulnerabilidades identificadas en el sistema, se hace necesario seleccionar una metodología de desarrollo seguro que permita integrar la seguridad a lo largo de todo el ciclo de vida del software. Existen diferentes enfoques reconocidos, entre los cuales destacan modelos como SDL (Security Development Lifecycle), OWASP SAMM y DevSecOps.

El modelo SDL propone un proceso estructurado en el que la seguridad se incorpora mediante fases definidas, incluyendo análisis de amenazas, revisiones de código y pruebas de seguridad. Por su parte, OWASP SAMM se enfoca en la madurez de las prácticas de seguridad dentro de una organización, proporcionando un marco para evaluar y mejorar procesos existentes. Sin embargo, ambos enfoques pueden resultar más formales o rígidos dependiendo del contexto del proyecto.

En contraste, DevSecOps propone una integración continua de la seguridad dentro de las prácticas de desarrollo y operación, permitiendo que los controles de seguridad se implementen de manera progresiva, automatizada y adaptable al entorno.

4.2 Metodología seleccionada: DevSecOps

A partir del análisis anterior, se selecciona la metodología DevSecOps como enfoque principal para el caso de estudio. Esta decisión se fundamenta en la necesidad de incorporar la seguridad de manera transversal en todas las fases del desarrollo, desde los requerimientos hasta el despliegue y mantenimiento del sistema.

DevSecOps no se limita a agregar controles de seguridad en etapas finales, sino que promueve una cultura en la que la seguridad es responsabilidad compartida entre desarrolladores, equipos de operaciones y especialistas en seguridad. Esto resulta especialmente pertinente en el sistema analizado, donde las fallas identificadas evidencian la ausencia de controles desde etapas tempranas.

4.3 Justificación de la elección frente al caso analizado

La elección de DevSecOps se justifica principalmente por su capacidad de adaptarse al tipo de sistema estudiado y a las debilidades detectadas. En primer lugar, el caso evidencia problemas originados desde el levantamiento de requerimientos, lo cual requiere una metodología que permita incorporar criterios de seguridad desde el inicio. DevSecOps facilita esta integración al incluir prácticas como análisis de requisitos de seguridad, revisión continua y automatización de controles.

En segundo lugar, el sistema presenta vulnerabilidades relacionadas con autenticación, validación de datos y control de acceso, las cuales pueden ser mitigadas mediante prácticas de integración continua, pruebas automatizadas y monitoreo constante, elementos propios del enfoque DevSecOps. Esto permite detectar errores de forma temprana, reduciendo el costo y el impacto de las fallas.

Otro aspecto relevante es la necesidad de mantener el sistema actualizado y seguro a lo largo del tiempo. A diferencia de metodologías más tradicionales, DevSecOps promueve un ciclo continuo de mejora, donde la seguridad no es una actividad puntual, sino un proceso permanente. Esto resulta fundamental en entornos como el sector salud, donde los riesgos evolucionan constantemente y requieren una respuesta dinámica.

4.4 Ventajas de DevSecOps en el contexto del sistema

La implementación de DevSecOps en el sistema analizado ofrece varias ventajas significativas. En primer lugar, permite integrar controles de seguridad desde la fase de requerimientos, evitando que las vulnerabilidades se originen desde el diseño. En segundo lugar, facilita la automatización de pruebas de seguridad, lo cual mejora la eficiencia del proceso de desarrollo y reduce la dependencia de revisiones manuales.

Asimismo, este enfoque promueve la colaboración entre los diferentes actores del proyecto, lo que contribuye a una mejor comprensión de los riesgos y a la implementación de soluciones más integrales. Finalmente, DevSecOps permite un monitoreo continuo del sistema en producción, lo cual es clave para detectar y responder a incidentes de manera oportuna.

En conclusión, la selección de DevSecOps como metodología de desarrollo seguro responde a la necesidad de adoptar un enfoque preventivo, continuo y adaptable frente a las vulnerabilidades identificadas en el sistema. Su capacidad para integrar la seguridad en todas las fases del ciclo de vida del software lo convierte en una opción adecuada para mejorar la calidad del sistema, reducir riesgos y garantizar la protección de la información en un entorno tan sensible como el sector salud.

5. Plan de integración desde los requerimientos.

5.1 Enfoque de integración de seguridad desde etapas tempranas

Una de las principales debilidades identificadas en el sistema analizado es la ausencia de criterios de seguridad desde la fase de requerimientos. Esto evidencia que las decisiones iniciales del desarrollo no consideraron aspectos fundamentales como la protección de datos, el control de acceso y la validación de la información. Por esta razón, el plan de integración propuesto se enfoca en incorporar la seguridad desde el inicio del ciclo de vida del software, alineado con el enfoque DevSecOps.

Integrar la seguridad desde los requerimientos implica no solo definir qué debe hacer el sistema, sino también cómo debe proteger la información y garantizar su uso adecuado. Esto permite prevenir vulnerabilidades en lugar de corregirlas posteriormente, reduciendo costos y mejorando la calidad del producto final.

5.2 Definición de requisitos de seguridad

En esta etapa se deben establecer requisitos funcionales y no funcionales relacionados con la seguridad. Para el sistema de citas médicas, estos requisitos incluyen aspectos como:

- Implementación de autenticación segura mediante almacenamiento cifrado de contraseñas
- Definición de roles y permisos para cada tipo de usuario
- Protección de datos sensibles mediante cifrado en tránsito (uso de HTTPS)
- Validación de entradas para evitar inyección de código
- Registro de eventos relevantes para auditoría

Estos requisitos deben documentarse desde el inicio y formar parte del alcance del sistema, evitando que la seguridad sea tratada como un elemento adicional o secundario.

5.3 Integración de seguridad en la fase de diseño

Una vez definidos los requerimientos, la seguridad debe reflejarse en el diseño de la arquitectura del sistema. En este caso, se propone una arquitectura que contemple separación de capas (presentación, lógica de negocio y datos), así como la implementación de mecanismos de autenticación y autorización robustos.

Asimismo, se deben definir políticas claras de acceso a la información, asegurando que cada usuario solo pueda visualizar y modificar los datos que le corresponden. El diseño también debe considerar la protección de sesiones, el manejo seguro de tokens y la restricción de acceso a recursos críticos.

5.4 Aplicación de prácticas seguras en el desarrollo

Durante la fase de desarrollo, el enfoque DevSecOps promueve la implementación de buenas prácticas de codificación segura. Esto incluye la validación de datos en el servidor, el uso de consultas parametrizadas para evitar inyección SQL y la protección de información sensible.

Además, se deben integrar herramientas que permitan realizar análisis de código y detectar vulnerabilidades de forma temprana. La automatización de estas revisiones contribuye a mantener un estándar de seguridad constante a lo largo del desarrollo.

5.5 Pruebas de seguridad e integración continua

El plan de integración contempla la ejecución de pruebas de seguridad como parte del proceso de desarrollo. Estas pruebas incluyen validaciones de autenticación, control de acceso y detección de posibles vulnerabilidades en la aplicación.

A través de prácticas de integración continua, se pueden automatizar pruebas cada vez que se realicen cambios en el código, permitiendo identificar errores antes de que el sistema pase a producción. Esto reduce significativamente el riesgo de fallas en entornos reales.

5.6 Seguridad en el despliegue y mantenimiento

La seguridad no finaliza con el desarrollo del sistema, sino que debe mantenerse durante su operación. En la fase de despliegue, es fundamental asegurar la correcta configuración del servidor, el uso de protocolos seguros y la protección de la base de datos.

Durante el mantenimiento, se deben aplicar actualizaciones constantes, monitorear el comportamiento del sistema y gestionar posibles incidentes de seguridad. DevSecOps facilita este proceso al promover un ciclo continuo de mejora y adaptación frente a nuevas amenazas.

El plan de integración propuesto demuestra que la seguridad puede y debe ser incorporada desde la fase de requerimientos, extendiéndose de manera transversal a todas las etapas del ciclo de vida del software. La adopción del enfoque DevSecOps permite transformar la seguridad en un proceso continuo, reduciendo la aparición de vulnerabilidades y fortaleciendo la confiabilidad del sistema.

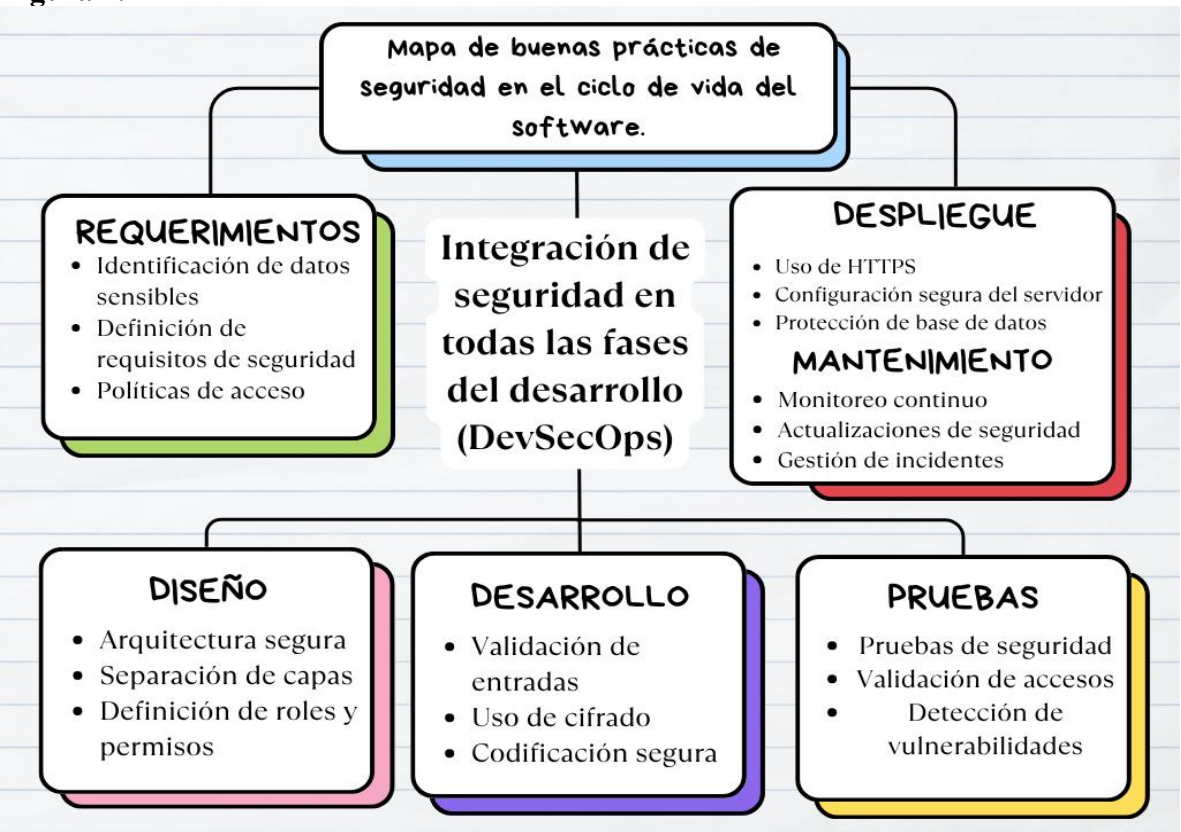
6. Mapa de buenas prácticas por fase del ciclo de vida.

6.1 Representación del mapa de buenas prácticas

Con el fin de visualizar de manera estructurada la integración de la seguridad en el ciclo de vida del software, se propone un mapa de buenas prácticas que relaciona cada fase del desarrollo con las acciones específicas orientadas a la protección del sistema.

Este mapa permite evidenciar cómo la seguridad no se limita a una etapa específica, sino que se incorpora de forma continua desde los requerimientos hasta el mantenimiento del sistema, en coherencia con el enfoque DevSecOps.

Figura 1.



Nota: Fuente propia.

6.2 Análisis del mapa

El mapa de buenas prácticas muestra que cada fase del ciclo de vida cumple un rol fundamental en la implementación de la seguridad. Desde la definición de requisitos hasta el mantenimiento del sistema, se establecen controles que permiten prevenir, detectar y mitigar vulnerabilidades.

En la fase de requerimientos, la seguridad se define como parte del alcance del sistema, mientras que en el diseño se traduce en una arquitectura adecuada. Durante el desarrollo y las pruebas, se implementan y validan los controles definidos, y finalmente, en el despliegue y mantenimiento, se garantiza la continuidad de la seguridad mediante monitoreo y actualización constante.

Este enfoque permite reducir significativamente los riesgos identificados en el sistema y fortalece la calidad del software desde una perspectiva integral.

7. Matriz metodológica con riesgos, soluciones y fases afectadas.

7.1 Enfoque de la matriz metodológica

Con el fin de relacionar las amenazas identificadas en el sistema con las soluciones propuestas y su integración dentro del ciclo de vida del software, se construye una matriz metodológica. Esta herramienta permite visualizar de manera clara cómo cada riesgo puede ser abordado mediante prácticas de seguridad específicas, alineadas con las fases del desarrollo y con el enfoque DevSecOps.

La matriz no solo facilita la organización de la información, sino que también evidencia la trazabilidad entre los problemas detectados y las acciones implementadas para mitigarlos, lo cual resulta fundamental en la gestión de la seguridad del software.

7.2 Matriz de integración de riesgos y soluciones

A continuación, se presenta la matriz metodológica construida a partir del análisis del caso:

Amenaza (STRIDE)	Riesgo identificado	Solución propuesta	Fase del ciclo de vida	Rol involucrado
Suplantación de identidad (Spoofing)	Acceso no autorizado por	Implementación de autenticación segura	Requerimientos / Desarrollo	Desarrollador / Seguridad

	credenciales débiles.	(hash de contraseñas, políticas de acceso).		
Manipulación de datos (Tampering)	Alteración de información en formularios o base de datos.	Validación de entradas y uso de consultas parametrizadas.	Desarrollo / Pruebas	Desarrollador
Repudio (Repudiation)	Falta de registro de acciones realizadas por usuarios.	Implementación de logs y auditoría de eventos.	Diseño / Desarrollo	Dev / Administrador
Divulgación de información (Information Disclosure)	Acceso a datos sensibles por usuarios no autorizados.	Control de acceso por roles y cifrado de datos.	Diseño / Desarrollo	Arquitecto / Dev
Denegación de servicio (DoS)	Saturación del sistema por múltiples solicitudes.	Implementación de control de tráfico y limitación de peticiones.	Despliegue / Mantenimiento	DevOps
Elevación de privilegios (Elevation of Privilege)	Acceso a funciones restringidas por mala gestión de permisos.	Definición estricta de roles y validación de permisos.	Diseño / Desarrollo	Arquitecto / Dev

7.3 Análisis de la matriz

La matriz metodológica evidencia que las amenazas identificadas en el sistema pueden ser mitigadas mediante la implementación de controles de seguridad distribuidos a lo largo del ciclo de vida del software. Se observa que la mayoría de los riesgos tienen su origen en fases tempranas como requerimientos y diseño, lo que refuerza la importancia de integrar la seguridad desde el inicio del desarrollo.

Asimismo, se destaca que las soluciones propuestas no dependen de una única fase o rol, sino que requieren la participación conjunta de diferentes actores, como desarrolladores, arquitectos y equipos de operaciones. Esto es coherente con el enfoque DevSecOps, en el cual la seguridad es una responsabilidad compartida y continua.

Finalmente, la matriz permite establecer una relación directa entre los riesgos y las acciones de mitigación, facilitando la toma de decisiones y el seguimiento de las medidas

implementadas. Esto contribuye a mejorar la calidad del sistema y a reducir la probabilidad de incidentes de seguridad en entornos productivos.

8. Conclusión

A partir del análisis realizado, se puede concluir que la mayoría de las vulnerabilidades presentes en el sistema no se originan únicamente en errores de implementación, sino en la ausencia de un enfoque de seguridad desde las etapas iniciales del desarrollo. La falta de definición de requisitos de seguridad, junto con debilidades en el diseño y la validación de la información, evidencian la necesidad de adoptar metodologías que integren la seguridad de manera estructurada.

La aplicación del modelo STRIDE permitió identificar y clasificar las principales amenazas del sistema, facilitando una comprensión clara de los riesgos asociados y su impacto en la confidencialidad, integridad y disponibilidad de la información. Este análisis fue clave para establecer una base sólida en la propuesta de soluciones.

En este sentido, la adopción de DevSecOps como metodología de desarrollo seguro representa una alternativa adecuada, ya que permite incorporar la seguridad de forma continua, promoviendo la automatización, la colaboración entre equipos y la detección temprana de vulnerabilidades. Su implementación no solo mejora la calidad del software, sino que también reduce los costos asociados a la corrección de errores en etapas avanzadas.

Finalmente, el desarrollo del mapa de buenas prácticas y la matriz metodológica permitió evidenciar cómo las amenazas pueden ser abordadas de manera integral a lo largo del ciclo de vida del software. Esto demuestra que la seguridad no debe considerarse como una fase independiente, sino como un proceso continuo que acompaña cada etapa del desarrollo, garantizando sistemas más confiables, seguros y alineados con las necesidades del entorno.

9. Referencias

9 best practices for secure coding in 2026. (2025, noviembre 10). Securityjourney.com. https://www.securityjourney.com/post/best-practices-for-secure-coding

Gottdiener, Z. (2022, enero 25). Gestión de riesgos en el desarrollo de software: 7 riesgos comunes. DOOR3. https://www.door3.com/es/blog/understanding-risk-management-in-software-development-7-common-risks

¿Qué es DevSecOps? (s/f). Redhat.com. Recuperado el 27 de marzo de 2026, de https://www.redhat.com/es/topics/devops/what-is-devsecops

¿Qué es el modelo de amenaza STRIDE? (2024, noviembre 26). Purestorage.com; Pure Storage. https://www.purestorage.com/es/knowledge/stride-threat-model.html

¿Qué es la Seguridad del Software y por qué es tan Importante Hoy? (2024, enero 1). Thales Cloud Security Products; Grupo Thales. https://cpl.thalesgroup.com/es/software-monetization/what-is-software-security

Seguridad del paciente. (s/f). Who.int. Recuperado el 27 de marzo de 2026, de https://www.who.int/es/news-room/fact-sheets/detail/patient-safety

Seguridad en el ciclo de vida de desarrollo del software. (s/f). Redhat.com. Recuperado el 27 de marzo de 2026, de https://www.redhat.com/es/topics/security/software-development-lifecycle-security

Thevarmannil, M. (2022, diciembre 7). What is the STRIDE Threat Model? Beginner's guide - 2026. Practical DevSecOps. https://www.practical-devsecops.com/what-is-stride-threat-model/

(S/f-a). Amazon.com. Recuperado el 27 de marzo de 2026, de https://aws.amazon.com/es/what-is/devsecops/

(S/f-b). Cloudflare.com. Recuperado el 27 de marzo de 2026, de https://www.cloudflare.com/es-es/learning/security/threats/owasp-top-10/

(S/f-c). Cloudflare.com. Recuperado el 27 de marzo de 2026, de https://www.cloudflare.com/es-es/learning/access-management/what-is-access-control/